In [ ]:
```
import sys
sys.executable
print(sys.executable)

version = sys.version
print(version)
```

```
c:\kalpesh\1.ITvedant\2.ML projects\IEX_Carbon_Credit_ML_Project\venv\Scripts\python.exe
3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
```

IEX Trading Intelligence DATA SCIENCE RESEARCH

1. INTRODUCTION

Project Objective

The objective of this machine learning system is to predict the performance of Instagram posts and support data-driven content planning for Graphura. Each post is classified into one of three performance categories based on historical engagement rate thresholds:

-High Performance -Medium Performance -Low Performance

1. Importing essential python libraries

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

import warnings
warnings.filterwarnings('ignore')
```

2. Dataset Loading

In [ ]:
```
from sqlalchemy import create_engine
```

In [ ]:
```
username = "root"
password = "root"
host = "localhost"
port = "3306"
database = "Carbon_Trading_DB"

engine = create_engine(f"mysql+pymysql://{username}:{password}@{host}:{port}/{database}")
```

In [ ]:
```
engine.connect()
```

Out[ ]:
```
<sqlalchemy.engine.base.Connection at 0x234c2c5b790>
```

In [ ]:
```
df = pd.read_sql("SELECT * FROM carbon_data", engine)
df.head()
```

Out[ ]:

	Company_ID	Industry_Type	Date	Energy_Demand_MWh	Fuel_Type	Emission_Produced_tCO2	Emission_Allowance_tCO2	Car
0	C099	Manufacturing	2025-03-20	2523.15	Renewable	1532.65	1428.65	
1	C015	Cement	2024-01-12	NaN	Renewable	1205.54	1228.54	
2	C047	Manufacturing	2024-08-28	977.27	Renewable	731.40	721.40	
3	C090	nan	2024-01-11	NaN	Natural Gas	1396.17	1258.17	
4	C095	Energy	2024-11-03	NaN	MIXED FUEL	1485.77	1533.77	

3. Data Cleaning & Preprocessing

In [ ]:
```
df.info()
df.isnull().sum()

df.columns = (df.columns.str.strip().str.lower().str.title())
```

```python
obj_cols = df.select_dtypes(include="object").columns

df[obj_cols] = df[obj_cols].apply(
    lambda col: col.astype(str)
                    .str.strip()
                    .str.lower()
                    .str.title()
)

for col in obj_cols:
    # remove spaces
    df[col] = df[col].astype(str).str.strip()

    # convert fake missing values to real NaN
    df[col] = df[col].replace(
        ["nan", "Nan", "NAN", "None","Unknown", "UNKNOWN", "null", ""],
        np.nan
    )

    # fill using mode
    df[col] = df[col].fillna(df[col].mode()[0])



# ---------------------------------------------------
# Select numeric columns
# ---------------------------------------------------
num_cols = df.select_dtypes(include=["number"]).columns

# ---------------------------------------------------
# Convert fake numeric strings to NaN (if any)
# ---------------------------------------------------
for col in num_cols:

    # Ensure numeric type (coerce converts invalid values → NaN)
    df[col] = pd.to_numeric(df[col], errors="coerce")

    # Replace infinite values (important for divisions)
    df[col] = df[col].replace([np.inf, -np.inf], np.nan)

    # Fill missing values using MEDIAN (robust to outliers)
    df[col] = df[col].fillna(df[col].median())



df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4511 entries, 0 to 4510
Data columns (total 14 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Company_ID               4511 non-null   object
 1   Industry_Type            4511 non-null   object
 2   Date                     4511 non-null   object
 3   Energy_Demand_MWh        4423 non-null   float64
 4   Fuel_Type                4511 non-null   object
 5   Emission_Produced_tCO2   4511 non-null   float64
 6   Emission_Allowance_tCO2  4511 non-null   float64
 7   Carbon_Price_USD_per_t   4425 non-null   float64
 8   Credits_Traded_tCO2      4511 non-null   float64
 9   Verification_Status      4511 non-null   object
 10  Compliance_Cost_USD      4511 non-null   float64
 11  Optimization_Scenario    4511 non-null   object
 12  Carbon_Cost_Savings_USD  4511 non-null   float64
 13  Transaction_Type         4511 non-null   object
dtypes: float64(7), object(7)
memory usage: 493.5+ KB
```

Out[ ]:

	Company_Id	Industry_Type	Date	Energy_Demand_Mwh	Fuel_Type	Emission_Produced_Tco2	Emission_Allowance_Tco2	Carb
0	C099	Manufacturing	2025-03-20	2523.15	Renewable	1532.65	1428.65	
1	C015	Cement	2024-01-12	1747.79	Renewable	1205.54	1228.54	
2	C047	Manufacturing	2024-08-28	977.27	Renewable	731.40	721.40	
3	C090	Manufacturing	2024-01-11	1747.79	Natural Gas	1396.17	1258.17	
4	C095	Energy	2024-11-03	1747.79	Mixed Fuel	1485.77	1533.77	

In [ ]:

```
from sklearn.impute import SimpleImputer

# Convert numeric columns stored as object
numeric_cols = ["Energy_Demand_Mwh","Carbon_Price_Usd_Per_T"]

for col in numeric_cols:
        df[col] = pd.to_numeric(df[col], errors="coerce")

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4511 entries, 0 to 4510
Data columns (total 14 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Company_Id               4511 non-null   object
 1   Industry_Type            4511 non-null   object
 2   Date                     4511 non-null   object
 3   Energy_Demand_Mwh        4511 non-null   float64
 4   Fuel_Type                4511 non-null   object
 5   Emission_Produced_Tco2   4511 non-null   float64
 6   Emission_Allowance_Tco2  4511 non-null   float64
 7   Carbon_Price_Usd_Per_T   4511 non-null   float64
 8   Credits_Traded_Tco2      4511 non-null   float64
 9   Verification_Status      4511 non-null   object
 10  Compliance_Cost_Usd      4511 non-null   float64
 11  Optimization_Scenario    4511 non-null   object
 12  Carbon_Cost_Savings_Usd  4511 non-null   float64
 13  Transaction_Type         4511 non-null   object
dtypes: float64(7), object(7)
memory usage: 493.5+ KB
```

In [ ]:

```
df.drop(columns=['Company_Id', 'Date'], inplace=True)
```

4.Feature Engineering`

In [ ]:

```
df.head()
```

Out[ ]:

	Industry_Type	Energy_Demand_Mwh	Fuel_Type	Emission_Produced_Tco2	Emission_Allowance_Tco2	Carbon_Price_Usd_Per_T
0	Manufacturing	2523.15	Renewable	1532.65	1428.65	35.00
1	Cement	1747.79	Renewable	1205.54	1228.54	34.69
2	Manufacturing	977.27	Renewable	731.40	721.40	35.00
3	Manufacturing	1747.79	Natural Gas	1396.17	1258.17	33.93
4	Energy	1747.79	Mixed Fuel	1485.77	1533.77	33.71

In [ ]:

```
# ==============================
# PART 1 : FEATURE ENGINEERING
# ==============================

import numpy as np
import pandas as pd

EPS = 1e-6

# ------------------------------
# 1. Emission Gap
# ------------------------------
df["Emission_Gap"] = (
    df["Emission_Produced_Tco2"]
```

```
    - df["Emission_Allowance_Tco2"]
)

# -------------------------------
# 2. Compliance Pressure
# -------------------------------
df["Compliance_Pressure"] = (
    df["Emission_Gap"]
    * df["Carbon_Price_Usd_Per_T"]
)

# -------------------------------
# 3. Cost per Energy Unit
# -------------------------------
df["Cost_per_MWh"] = np.where(
    df["Energy_Demand_Mwh"] > 0,
    df["Compliance_Cost_Usd"] /
    (df["Energy_Demand_Mwh"] + EPS),
    0
)

ratio_cols = ["Cost_per_MWh"]

for col in ratio_cols:
    lower = df[col].quantile(0.01)
    upper = df[col].quantile(0.99)
    df[col] = df[col].clip(lower, upper)

# -------------------------------
# 4. Fuel Carbon Factor
# -------------------------------
fuel_factor = {
    "Coal": 0.95,
    "Mixed Fuel": 0.75,
    "Natural Gas": 0.5,
    "Renewable": 0.25
}

df["Fuel_Carbon_Factor"] = (
    df["Fuel_Type"]
    .map(fuel_factor)
    .fillna(0.5)
)



# -------------------------------
# 5. Target Encoding
# -------------------------------
target = "Transaction_Type"

target_map = {
    "Buy": 0,
    "Hold": 1,
    "Sell": 2
}

df[target] = df[target].map(target_map)
```

Feature Selection

In [ ]:

```
# ================================
# PART 2 : MODEL FEATURES
# ================================

# Columns causing leakage
leakage_cols = [
    'Compliance_Cost_Usd',
    'Carbon_Cost_Savings_Usd'
]

# Numerical features (FIXED)
num_cols = [
    "Energy_Demand_Mwh",
    "Carbon_Price_Usd_Per_T",
    'Credits_Traded_Tco2',

    # Engineered Features
    "Emission_Gap",
    "Compliance_Pressure",
    "Cost_per_MWh",
```

```
    "Fuel_Carbon_Factor"
]

cat_cols = ["Optimization_Scenario"]


target = "Transaction_Type"
```

In [ ]:
```
from sklearn.model_selection import train_test_split

xc = df[num_cols + cat_cols]

yc = df[target]

xctrain, xctest, yctrain, yctest = train_test_split(xc,yc,test_size=0.2,random_state=42,stratify=yc)


print("Feature shape:", xc.shape)
print("Target shape:", yc.shape)
```

```
Feature shape: (4511, 8)
Target shape: (4511,)
```

In [ ]:
```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import RobustScaler, OneHotEncoder


# ---------- Numerical Pipeline ----------
num_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),  # better for skewed data
    ("scaler", RobustScaler())
])


# ---------- Categorical Pipeline ----------
cat_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(
        handle_unknown="ignore",
        sparse_output=True
    ))
])



# ---------- Preprocessor ----------
preprocessor = ColumnTransformer(
    transformers=[
        ("num", num_pipeline, num_cols),
        ("cat", cat_pipeline, cat_cols),
    ],
    remainder="drop"   #  MUST be inside ColumnTransformer
)
```

CLASSIFICATION MODELS (post performance category)

Model Training, Model Evaluation & Model Training

In [ ]:
```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

def evaluate_model(model, xctest, yctest, name):

    # ---- labels for classes ----
    labels = ['Buy', 'Sell', 'Hold']   # change order if needed

    preds = model.predict(xctest)
    acc = accuracy_score(yctest, preds)

    print(f"\n{name}")
    print("Accuracy:", acc)
    print(classification_report(yctest, preds, target_names=labels))

    # Confusion Matrix
    cm = confusion_matrix(yctest, preds)
    print("Confusion Matrix:")
    print(cm)
```

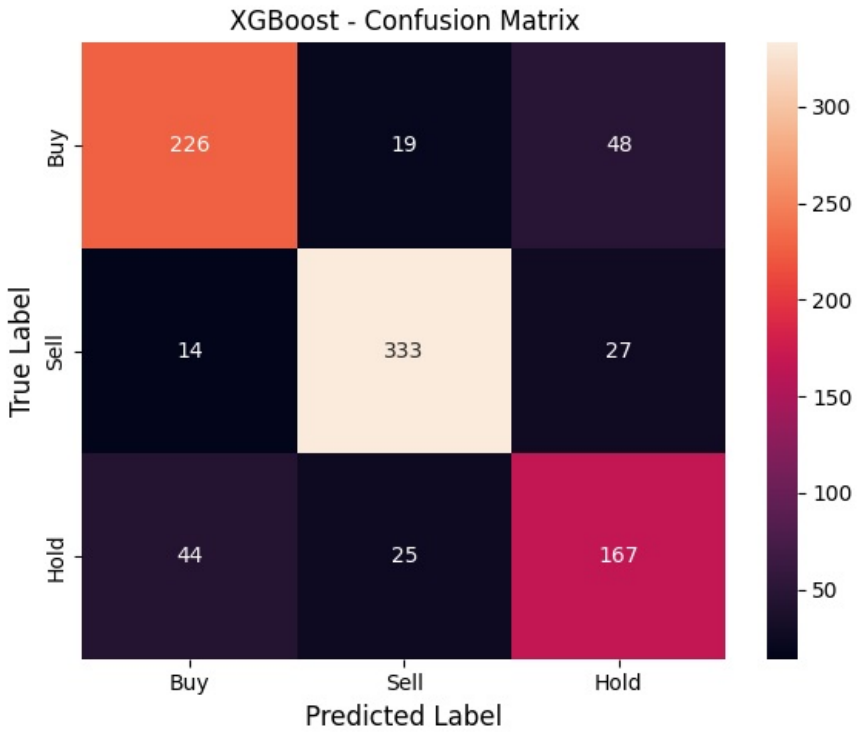
```
    # ---- Plot ----
    plt.figure(figsize=(6,5))
    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        xticklabels=labels,
        yticklabels=labels
    )

    plt.title(f"{name} - Confusion Matrix")
    plt.xlabel("Predicted Label", fontsize=12)
    plt.ylabel("True Label", fontsize=12)
    plt.tight_layout()
    plt.show()

    return acc
```

In [ ]:
```
from sklearn.pipeline import Pipeline
```

Best Model:- XG Boost

In [ ]:
```
from xgboost import XGBClassifier

xgb_pipe = Pipeline([
    ("preprocessing", preprocessor),
    ("model", XGBClassifier(
        n_estimators=300,
        max_depth=6,
        learning_rate=0.03,
        subsample=0.85,
        colsample_bytree=0.85,
        gamma=0.1,
        reg_lambda=2,
        reg_alpha=0.2,
        eval_metric="mlogloss",
        tree_method="hist",   # faster training
        random_state=42
    ))
])

best_xgb = xgb_pipe.fit(xctrain, yctrain)

evaluate_model(best_xgb, xctest, yctest, "XGBoost")
```

```
XGBoost
Accuracy: 0.8039867109634552
              precision    recall  f1-score   support

         Buy       0.80      0.77      0.78       293
        Sell       0.88      0.89      0.89       374
        Hold       0.69      0.71      0.70       236

    accuracy                           0.80       903
   macro avg       0.79      0.79      0.79       903
weighted avg       0.80      0.80      0.80       903

Confusion Matrix:
[[226  19  48]
 [ 14 333  27]
 [ 44  25 167]]
```



Out[ ]: 0.8039867109634552

In [ ]:
```
best_model = best_xgb  # XGBoost performed best in this example
```

In [ ]:
```
EXPECTED_COLUMNS = xctrain.columns.tolist()
def safe_input(df):
    for col in EXPECTED_COLUMNS:
        if col not in df:
            df[col] = None
    return df[EXPECTED_COLUMNS]
```

Saving the model for deployment

In [ ]:
```
import os, joblib

print(os.getcwd())
os.makedirs("models", exist_ok=True)
joblib.dump(best_model, "best_model_pipeline.joblib")
print(os.listdir("models"))
```

```
c:\kalpesh\1.ITvedant\2.ML projects\IEX_Carbon_Credit_ML_Project
[]
```

In [ ]:
```
import sklearn
print(sklearn.__version__)
```

```
1.4.2
```

In [ ]:
```
import sys
print(sys.version)
```

```
3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
```

In [ ]:
```python
import sys, sklearn, joblib

print(sys.version)
print(sklearn.__version__)

joblib.dump(best_model, "best_model_pipeline.joblib")
```

```
3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
1.4.2
```

Out[ ]:
```
['best_model_pipeline.joblib']
```

In [ ]: